

Javaのメモ

- 配列の定義(new)
- 先頭アドレス
- ガーベジコレクション
- 多次元の配列
- 例外の種類(エラーの種類)
- メソッドの定義
- オーバーロード
- 引数に配列を用いる
- コマンドライン引数

配列の定義(new)

変数valueに配列scoresの要素を一つずつ入れていく

```
int scores = {1,2,3,4,5,6,7,8,9,10};  
for(int value : scores){}
```

先頭アドレス

配列arrayAのarrayAには配列の最初のアドレスが格納されている

```
int[] arrayA = {1,2,3};
```

arrayBにarrayAの先頭アドレスの情報が格納される。出力する際は100が出力される。

```
arrayB = arrayA;  
arrayB[0] = 100;  
System.out.println(arrayA[0]);
```

ガーベジコレクション

Javaはメモリを使い終わったら自動でメモリをクリアしてくれる機能がある

Javaの配列にnullを代入したらそのあとから、array(参照元)は消去されるので代入できない。

```
array = null;  
array[0] = 10;
```

実行後エラー

多次元の配列

二次元の配列の定義

```
// 例：データ型 [][] 変数名 = new データ型 [行数][列数];  
int [][] arr = new int[5][5];
```

こうすると5行5列の配列ができる

ほかにも最初から値を代入してくと、このような定義式になる

```
//Example1  
int[][] scores = {{40,50,60},{80,60,70}};  
  
//Example2  
int[][] scores = {  
    {40,50,60},  
    {80,60,70}  
};
```

※この場合scores[0]には{40,50,60}の先頭アドレスが、scores[1]には{80,60,70}の先頭アドレスが入っている

例外の種類（エラーの種類）

- 「配列がないですよ」

```
NullPointerException:
```

- 「配列の外を参照していますよ」

※Boundsは境界という意味。なのでOutOfBoundsは境界の外に出ますよ～という意味

```
ArrayIndexOutOfBoundsException
```

メソッドの定義

クラスメソッド

hello("引数 1 ")の引数 1 のことを実引数

public static void hello("引数 2 ")の引数 2 のことを仮引数という

```
//クラスメソッド
public class Main{
    //メインメソッド
    public static void main(String[] args){
        hello("秀伍");
    }

    //helloメソッド(オリジナルメソッド)
    public static void hello(String name) {
        System.out.println(name + "さん、こんにちは");
    }
}
```

Bash

秀伍さん、こんにちは

戻り値について

returnを使うことで戻り値を使える >> ansに代入される

```
public class Main{
    public static void add(int x,int y){
        int ans = x + y;
        return ans;
    }

    public static void main(String[] args){
        int ans = add(100,10);
        System.out.println("100 + 10 = " + ans);
        // ("100 + 10 = " + add(100,10))と書いてもよい
    }
}
```

Bash

100 + 10 = 110

オーバーロードとは

多重定義のこと

仮引数（の数、型）が違えば同じ名前のメソッドを複数定義できる

```
public static int add(int x, int y) {  
    return x + y;  
}  
  
public static int add(int x, int y, int z) {  
    return x + y + z;  
}  
  
public static void main(String[] args) {  
    // TODO 自動生成されたメソッド・スタブ  
    System.out.println("10+20=" + add(10,20));  
  
    System.out.println("10+20+30=" + add(10,20,30));  
  
}
```

Javaは同じメソッド名でコンパイラが勝手にどのメソッドを使うか認識してそれぞれのメソッドに割り当ててくれる

引数や戻り値に配列を用いる

引数に持ってきた配列arrayにはその参照している配列の先頭アドレスが入っている

```
//配列を自動で作成して返回值(配列の先頭アドレス)を返す  
public static int[] makeArray(int size) {  
    int[] newArray = new int[size];  
  
    for (int i = 0; i < newArray.length; i++) {  
        newArray[i] = i;  
    }  
  
    return newArray;  
}  
  
// ここでarrayはmakeArrayメソッドのnewArrayの先頭アドレスを参照できる  
int[] array = makeArray(3);  
arrayを出力
```

Bash

```
0,1,2
```

```
//引数として渡された配列内の要素全てをインクリメント  
public static void incArray(int[] array) {  
    for (int i = 0; i < array.length; i++) {
```

```
        array[i]++;
    }
}

//メイン（返り値なしでincArray実行後、array内の要素を出力）
public static void main(String[] args) {
    int[] array = {1,2,3};
    incArray(array);
    for (int i : array) {
        System.out.println(i);
    }
}
```

Bash

2,3,4

コマンドライン引数

public static void main(String[] args)の引数argsのこと。このargsは文字型として引き渡される

Bash

```
java Example.java 引数1 引数2 引数3
```

上記を実行後、以下ようになる

```
public static void main(String[] args) {
    args[0] -> "引数 1 "
    args[1] -> "引数 2 "
    args[2] -> "引数 3 "
    args[3] -> NULL
}
```

クラスメソッドを呼ぶ

外部ファイルからクラスメソッドを呼ぶ

別のクラスで定義されているクラスメソッド(static宣言がついたメソッド)を呼び出すときは、クラス名.メソッド名()の形で呼び出す。

FileA

```
int total = CalcLogic.tasu(a,b);  
int delta = CalcLogic.hiku(a,b);
```

FileB

```
public class CalcLogic {  
    public static int tasu( int a, int b) {  
        return (a + b);  
    }  
    public static int hiku(int a, int b ) {  
        return (a - b);  
    }  
}
```

今まで作ったファイルはどこに行ったのか

Eclipseがあるフォルダーの自分で作ったフォルダーの中にsrcディレクトリとbinディレクトリがある

- srcディレクトリ

ソースコードファイル

- binディレクトリ

バイナリファイル(実行ファイル)

Eclipseのパッケージの作り方

ファイル -> 新規 -> パッケージ
パッケージを右クリックしてさらにパッケージで
packageから
package.tool1のように
階層構造を作れる

importについて

Javaのcalcapp.logics.CalcLogicsのような長いパッケージ名を短くする

```
import calcapp.logics.CalcLogic;
int total = CalcLogic.tasu(a,b);
```

このようにimportを作ることによって長いパッケージ名を省略できる

java.utilパッケージとは

javaにとって役に立つパッケージを集めたもの

- java.lang ...欠かせない便利なもの
- java.util ...便利にするための様々なツール
- java.math ...数学系
- java.net ...ネットワーク系
- java.io ...ファイル読み書き系

javaのパッケージ調べ方

java パッケージ リファレンスと調べれば出てくる

クラス定義

```
public class Hero {
    String name; //ここでmainメソッド外で作る変数のことを「フィールド」という
    int hp;
    public void attack() {}
    public void sleep() {}
    public void sit(int sec) {}
    public void slip() {}
    public void run() {}
}
```

※基本的にフィールド名は小文字を作る (example_Sum)みたいな？

コンストラクタ

- クラス名と同じ名前のメソッド
- 戻り値の宣言しない(voidも×)
- インスタンス生成時に自動的に呼び出される
- コンストラクタを一つも定義しないとデフォルトコンストラクタ(引数なし、中身は空っぽ)が自動的に定義される

thisは自分自身のメソッドを意味し、Hero()で呼び出した場合、上のメソッドが呼び出される

```
public Hero(String name) {
    this.hp = 100;
    this.name = name;
}
```

```
public Hero() {
    this("ダミー");
}
```

継承

あるクラスのフィールドやメソッドを引き継いだクラスを定義すること

```
public class SuperHero extends Hero { //このSuperHero extendsが継承するためのものである
    boolean flying;
    public void fly() {
        this.flying = true;
        System.out.println("飛び上がった！！");
    }
    public void land() {
        this.flying = false;
        System.out.println("着地した！！");
    }
}
```

Human----Enemy
|
Human Type Monster
のような複数形賞をすることはできない。

Override

親クラスのメソッドを子クラスで再定義すること

- オーバーロード

```
public run(int a, int b) {
    return a+b;
}
public run(int a) {
    run(a,5);
}
```

このように同じメソッド名でもそれぞれ機能するのがオーバーロード

- オーバーライド

親クラスのメソッドを子クラスで再定義する


```
public class Hero {
    String text = "Hello World";
    public void run() {
        System.out.println(text);
    }
}

public class SuperHero extends Hero { //extends Hero はここでHeroクラスの性質を持ってきた
    //のが「継承」という。今回はtext = "Hello World"を持ってきた
    String name = "藤原秀伍";
    public void run() {
        System.out.println(text + name); //Heroクラスにあるけど、こっちで再定義することをオーバー
        ライドという
    }
}
```

継承とオーバーライドの禁止

- クラスの宣言にfinalをつける > 継承を禁止できる
- メソッドの宣言にfinalを付ける > オーバーライド禁止できる
- 変数宣言にfinalをつける > 書き換えできない変数(定数)

```
public final class Hero {} //クラス
public final Jump() {} //メソッド
final int number = 20; //変数
```

子クラス(継承したクラス)が親クラスのメソッドを使うには

```
super.run();
```

これは親クラスで定義されたrun()を実行する

親クラスのコンストラクタを継承するには

```
class Parent {
    Parent() {
        System.out.println("親クラスのコンストラクタ");
    }
}

class Child extends Parent {
    Child() {
        super();
        System.out.println("子クラスのコンストラクタ");
    }
}
```

```
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Child child = new Child();  
    }  
}
```

Bash

親クラスのコンストラクタ
子クラスのコンストラクタ

継承の注意点

コンストラクタは継承されない

コンストラクタとはそのクラスの初期化のことを意味するので、子クラスが親クラスの初期化を持ってきたらそれは初期化とはいかない

抽象クラス

- インスタンス化できない
- 継承させるためのクラス
- abstractを付けて宣言する
- 抽象メソッドを持つクラスは抽象クラスとして宣言する必要あり

抽象クラス（親クラス）にはある書き方をしなければならない。

抽象クラスには**public abstract class**と定義しないとイケない。そして抽象クラスのメンバに例えば、**public abstract void run**とすると、runメソッドは抽象メソッドとなり、子クラスで定義しないとイケない

```
//abstractがついているので、抽象化したメンバをオーバーライド  
public abstract class Character {  
    public abstract void run() {  
        System.out.println("走ります");  
    }  
    public void attack() {  
        System.out.println("やりますねえ");  
    }  
}  
  
//この子クラスの親クラスは、抽象クラスなので必ず抽象メソッドをオーバーライドしないとイケない  
public class Hero extends Character {  
    public void run() {
```

```
        System.out.println("勇者は走ります");  
    }  
}
```

インターフェース

抽象度が高い抽象クラスを、インタフェースとして特別に扱うことができる

インタフェースとは？

- 定義では「public interface クラス名」
- 継承では「public class クラス名 implements インタフェース名」

```
//インタフェースとして定義する  
public interface CleaningService{  
  
//インタフェースを継承する場合の子クラスの定義  
public class KyotoCleaningShop implements CleaningService{
```

例

```
public interface CleaningService {  
    Shirt wasShirt(Shirt s); //これら三つは子クラスで必ず定義する必要がある  
    Towl wasTowl(Towl t);  
    Coat washCoat(Coat c);  
}
```

多重継承

インタフェースは多重で継承できる。唯一ここで多重継承が可能

もしも継承する場合は、全ての引数がインタフェースである必要がある

```
//定義 -> implements インタフェース1 インタフェース2 インタフェース3  
public class KyotoCleaningShop implements CleaningService MoneyShop WashShop {}
```

インタフェースを継承して子インタフェースのようなものを作る

```
public interface Creature{  
public interface Human extends Creature {}
```

多態性

オブジェクト指向プログラミングを支える三大機能の一つで、多様性やポリモーフィズムといわれる。

あるものを、あえてざっくりとらえることで、様々なメリットを享受できる機能

```
//SuperHeroクラスの親のクラス = Characterクラス
SuperHero h = new Super Hero();

Character c = new SuperHero();
//親クラス型の変数に子クラスのインスタンスを代入し、親クラスのインスタンスとして利用できる。
```

どんなメリットがあるのか？

以下のようなクラスがあった場合は、ポリモーフィズムという、ざっとAnimalクラスだよね。とまとめやすくなる。

```
class Animal {
    void makeSound() {
        System.out.println("Some generic animal sound");
    }
}

class Dog extends Animal {
    @Override //@Overrideとすることで無事にオーバーライドしたか確認する機能を付与できる
    void makeSound() {
        System.out.println("Woof Woof!");
    }
    void attack() {
        System.out.println("Dogの攻撃！！");
    }
}
```

- Dog mydog = new Dogの場合

```
Dogクラスの全てのメソッドを使うことができる
mydog.makeSound 可能
mydog.attack     可能
```

- Animal mydog = new Dogの場合

```
Animalクラスで定義されたメソッドを使うことができる  
mydog.makeSound 可能  
mydog.attack     実行不可
```

キャスト

親のクラス型から子のクラス型へキャスト（型変換）できる。（ダウンキャストと呼ぶ）

instanceof演算子とは

キャストできるか判定する演算子

```
if( c instanceof SuperHero){  
    SuperHero h = (SuperHero)c;  
    h.fly();  
}  
//cがSuperHeroから型変換（キャスト）できますよなら、true  
//キャストできないならfalse  
//結果：-> CharacterはSuperHeroからキャストできない
```

上記の省略

```
if(c instanceof SuperHero h) {  
    h.fly();  
}  
//上と同じ実行結果がある
```

カプセル化

カプセル化はアクセス制御をし、不具合を事前に見つけるプログラム

getterとsetter

getterはNameを直接参照せずに名前を入手する

```
public String getName() {  
    return this.name;  
}
```

setterは値を直接書き込まずに代入できる

```
public void setName( String name ) {  
    this.name = name;  
}
```

getやsetをつけるかは習慣づけされているので、必ずではない

getter,setterのメリット

フィールドへのアクセスを制限

```
public void setName(String name) {  
    if(name == null) {  
        throw new IllegalArgumentException("名前が未定義である。処理を中断"); //エラーを出  
        して、強制終了する  
    }  
    if(name.length() <= 1) {  
        throw new IllegalArgumentException("名前が短すぎる。処理を中断");  
    }  
}
```

2種類の制限

public -> 全クラスが共有可能で緩い

package public -> パッケージ内のクラスが共有可能でやや緩い

APIの活用

暗黒の継承

Objectクラス

全てのクラスの最上位のクラス。Objectクラスのメソッド

toString()メソッド
equals()メソッド

=>全てのクラスはこのメソッドを継承する
※つまり何のメソッドを定義していないクラスでも.toString()が機能する

- equals() ... あるインスタンスと自分自身が同じか調べる

- toString() ... 自分自身の内容の文字列表現を返す

toStringとequalsを変更する

toStringとequalsの仕様を変更するにはオーバーライドをすればよい

```
//toStringの仕様変更

public String toString() {
    return "名前：" + this.name + "/HP：" + this.hp;
}
```

```
//equalsの仕様変更

public boolean equals(Object o) {
    if(this == o) {return true;}
    if(o instanceof Hero h) {
        if(this.name.equals(h.name)) {
            return true;
        }
    }
}
```

※toString()はメソッド名を省略したら呼び出されるメソッド

staticによるフィールド共有

staticを使用すると、全てのオブジェクト（インスタンス）がそのクラス変数を共有できる。

クラス名.静的フィールドの形で参照できる（インスタンスを生成しなくても）

```
//定義
public class Hero{
    String name;
    int hp;
    static int money;
}
//参照方法 -> クラス名.静的フィールド名
//静的フィールドをクラス変数とも呼ぶ
Hero.money
```

文字列調査メソッド

- .length()...全角も半角も一文字として文字数を数える
- .equals(object o)...内容が等しいかどうか調べる
- .equalsIgnoreCase(String s)...大文字小文字区別なしで内容が等しいかどうか調べる

- `.isEmpty()`...空文字かどうか調べる

文字列検索メソッド

- `.contains( String s )`...一部に文字列sを含むか調べる
- `.startsWith( String s )`...文字列sで始まるか調べる
- `.endsWith( String s )`...文字列sで終わるかどう調べる
- `.indexOf( int ch )` `.indexOf( String str )`...文字chが最初に登場する位置を調べる
- `.lastIndexOf( int ch )` `.lastIndexOf( String str )`...文字ch文字列strを後ろから検索して最初に登場する位置を調べる

文字列変換メソッド

- `.toLowerCase()`...大文字を小文字に変換する
- `.toUpperCase()`...小文字を大文字に変換する
- `.trim()`...前後の空白を除去する
- `.replace( String before, String after )`...文字列を置き換える

`new`でインスタンスを作る際、コンピュータへの負荷が大きい

StringBufferクラス

文字列を格納するバッファを持つ文字列捜査のためのクラス

`append`メソッドで文字列をバッファに追加できる

```
public class Main {  
  
    public static void main(String[] args) {  
        // TODO 自動生成されたメソッド・スタブ  
        StringBuilder sb = new StringBuilder(); //この型により、バッファにため込んでいく  
        for(int i = 0; i < 10000; i++) {  
            sb.append("Java");  
        }  
        String s = sb.toString();  
        System.out.println(s);  
    }  
}
```

正規表現

パターンマッチング

```
public boolean example(String name) {  
    //最初にA~Zで始まり、そのあとにA~Z0~9である文字の文字列が7文字分。返り値は真か偽  
    return name.matches("[A-Z][A-Z0-9]{7}");  
}
```

その文字でなければならない

```
//必ずJavaが入っていないといけない  
name.matches("Java");
```

任意の一文字の0回以上

```
"あいう00xx019".matches(".*") //true
```

〇〇で始まる任意の文字列

```
// Maで始まる任意の文字列  
s.matches("Ma.*")  
// fulで終わる任意の文字列  
s.matches(".*ful")
```

\\の文字の判定

\\はjavaのmatchesメソッドでは特殊な文字のため、「\\」を判定するときは以下のようにしないといけない

```
// \\で始まる任意の文字列  
if( s.matches("\\\\\\\\\\.*")) {  
    処理  
}
```

指定回数の繰り返し

```
{n} ... 直前の文字のn回の繰り返し  
{n,} ... 直前の文字のn回以上の繰り返し  
{n,m} ... 直前の文字のn回以上m回以下の繰り返し  
? ... 直前の文字の0回または1回の繰り返し  
+ ... 直前の文字の一回以上の繰り返し
```

既に定義された文字クラス

```
\d ... いずれかの数字([0-9])  
\w ... いずれかの英語・数字・アンダーバー([0-9a-zA-Z_])  
\s ... 空白文字(スペース文字、タブ文字、改行文字)
```

ハットとダラー

```
^○ ... 先頭文字列が○である  
○$ ... 最後尾文字が○である  
^j.*p$ ... jで始まってpで終わる任意の文字列
```

文字列の分割

```
String s = "abc,def,:ghi";  
String[] words = s.split("[,:]");  
words -->[abcdefghi]
```

文字列の置換

```
String s = "abc,def,:ghi";  
String w = s.replaceAll("[beh]", "X"); //bとeとhをXにしますよ。という意味  
System.out.println(w); // aXc,dXf,:gXi
```

文字列の書式整形

桁をそろえたりする機能 -> フォーマット指定詞

```
String.format("%d日で%sわかる%s入門", 3, "すっきり", "Java");
```

可変長引数

可変長引数とは、引数の数を調整することができる技術

```
public static void methodA( String...args) {  
    for( String s: args ) {  
        System.out.println(s);  
    }  
}
```

```
public static void main( String[] args) {  
    methodA("aaa","bbb","ccc");  
}
```

出力

```
aaa  
bbb  
ccc
```

日付と時刻の指定

処理時間の計測

```
public class Main {  
    long start = System.currentTimeMillis();  
    long end = System.currentTimeMillis();  
    System.out.println("処理にかかった時間は..." + (end - start) + "ミリ秒でした");  
  
    //currentTimeMillisが起動した時点での時間帯を記録  
}
```

日付のデータを扱うライブラリ

日付のデータの方式を格納する

```
import java.util.Date;  
  
public class Main {  
    public static void main( String[] args) {  
        Date now = new Date();  
        System.out.println(now); //外国の日付形式で表示される  
        System.out.println(now.getTime()); //数値データのみで表示される  
        Date past = new Date(1600705425827L);  
        System.out.println(past)  
    }  
}
```

※注意 : `java.sql.Date()`はデータベース専用の日付ライブラリであり、混同してはならない

Dateインスタンスを使って日付データを処理する - Calenderインスタンスで日付データを処理する

```
import java.util.Calendar;

//インスタンスの取得
Calendar c = Calendar.getInstance();

//日付の設定
c.set(年,月,日,時,分,秒);
c.set(Calendar.MONTH, 9); //ここでは月を9月に変更している

//インスタンス変数cから日付を持ってくる
Date d = c.getTime();
System.out.println(d);
```

```
import java.util.Calendar;
import java.util.Date;

Date now = new Date(); //nowという日付変数を作成
Calendar c = Calendar.getInstance();

// 時間帯の書式をセットする
c.setTime(now);
int y = c.get(Calendar.YEAR);
System.out.println("今年は" + y + "年です");
```

SimpleDateFormat

```
import java.text.SimpleDateFormat;
import java.util.Date;

public class Main {

    public static void main(String[] args) throws Exception{ //このメソッドの中で発生した
エラーを呼び元へ投げる（呼び元で処理してもらうことができる）

        // TODO 自動生成されたメソッド・スタブ
        SimpleDateFormat f = new SimpleDateFormat("yyyy/MM/dd HH:mm:ss");

        //文字列からDateインスタンスを生成
        Date d = f.parse("2023/09/18 05:53:20");
        System.out.println(d);

        //Dateインスタンスから文字列を生成
        Date now = new Date();
        String s = f.format(now);
        System.out.println("現在は" + s + "です");
    }
}
```

TimeAPI

```
//Instantの生成
Instant i1 = Instant.now();

//Instantとlong値との相互変換
Instant i2 = Instant.ofEpochMilli(1600705425827L);
long l = i2.toEpochMilli();

//ZonedDateTimeの生成方法
ZonedDateTime z1 = ZonedDateTime.now();
ZonedDateTime z2 = ZonedDateTime.of(2020,1,2,3,4,5,6,ZoneId.of("Asia/Tokyo"));

//InstantとZoneDateTimeの相互変換
Instant i3 = z2.toInstant();
ZonedDateTime z3 = i3.atZone(ZoneId.of("Europe/London"));

//ZonedDateTimeの利用方法
System.out.println("東京" + z2.getYear() + z2.getMonth() + z2.getDayOfMonth() +
z2.getHour() + z2.getMinute());
System.out.println("ロンドン" + z3.getYear() + z3.getMonth() + z3.getDayOfMonth() +
z3.getHour() + z3.getMinute());

if (z2.isEqual(z3)) {
    System.out.println("これらは同じ瞬間を指しています");
}
```

- Dateクラス .. Instantクラス(ナノ秒単位) -> その瞬間の時刻データを詳しく記録
- Calenderクラス .. ZonedDateTimeクラス -> ユーザに見やすいようなデータ形式にする

LocalDateTimeについて

```
//LocalDateTimeの生成方法
LocalDateTime l1 = LocalDateTime.now();
LocalDateTime l2 = LocalDateTime.of(2020,1,1,9,5,0,0);

//LocalDateTimeとZoneDateTimeの相互変換
ZonedDateTime z1 = l2.atZone(ZoneId.of("Europe/London"));
LocalDateTime l3 = z1.toLocalDateTime();
```

- LocalDateTimeクラス .. あいまいな時間の情報を扱うため、メモとして最適
- ZonedDateTime .. タイムゾーンを扱うので、厳密にナノ単位での時間情報が必要

日付のメソッド一覧

```
.plusDays(日数) ... そのオブジェクトの日付に日数分加算する
.format(フォーマット) ... そのオブジェクトの日付を指定した日付の形式にする
DateTimeFormatter fmt = DateTimeFormatter.ofPattern("yyyy/MMM/dd")
... フォーマットを作れる。この場合、fmt(変数)に'yyyy/MM/dd'のフォーマットが入っている
.now() ... 現在の時刻を取得する
Period p1 = Period.ofDays(3) ... 3日"間"を生成するクラス (Periodクラス)
Period.between(d1,d2) ... d1とd2の間の日数を入れる。
```

コレクションとは

複数のデータをまとめて扱う仕組み

- List - 順序通りに並べて格納する
 - **ArrayList**
 - **LinkedList**
- Set - 順序があるとは限らない
 - **HashSet**
 - **LinkedHashSet**
- Map - ペアで対応付けて格納する
 - **HashMap**
 - **LinkedHashMap**

Listの書き方

配列を使ったコード

```
//配列を準備
String[] names = new String[3];

//3人を追加
names[0] = "たなか";
...
```

ArrayListを使ったコード

```
import java.util.ArrayList;

//ArrayListを準備
```

```
ArrayList<String> names = new ArrayList<String>(); // <型>この書き方をジェネリックスと呼ぶ

//3人を追加
names.add("たなか");
...
```

ジェネリックスについて

ジェネリックスの書き方

ジェネリックスの特徴

ArrayList<型>

型の場所にはインスタンスでも格納することができる。

要素の追加

```
変数名.add(追加したいデータ);
```

取り出し

```
変数名.get(インデックス番号);
```

ラッパークラス

ジェネリックスにインスタンスが入るものの、普通のデータ型は入らない。普通のデータ型は以下の方法で

```
// intデータを格納できるListArray -> ラッパークラスの定義
import java.util.Integer;

//int型のListArrayを定義
ArrayList<Integer> = new ArrayList<Integer>();

//データの追加
ArrayList.add(10);
```

イテレーター

リストの中身を一つずつ取り出す方法のひとつで、コレクションクラスの中身を順に取り出すための専用道具。「次へ」と指定すれば、自動で進んでくれる。

```
// イテレーターの取得
Iterator<リスト要素の型> it = リスト変数.iterator();
// 例

//データの作成
ArrayList<String> names = new ArrayList<String>();
names.add("湊");
names.add("朝香");

//イテレーターの作成
Iterator<String> it = names.iterator();

//イテレーターの操作
while(it.hasNext()) { //hasNextは、次があるかどうか確認するメソッドである
    String e = it.next(); //序盤はrootから始まるから、nextを指定する
    System.out.println(e);
}
```

Linkedリスト

普通のリストは並べられたポインタのアドレス順にデータを格納するが、Linkedリストはいろいろなアドレス先に転移したリストの塊である。

```
//ポリモーフィズムで、まとめられている
List<String> list1 = new ArrayList<String>();
List<Hero> list2 = new LinkedList<Hero>();
```

java.util.HashSetクラス

集合（セット）というデータ構造を表す。集合は重複がなく、順序を持たない複数の情報を格納できる。

```
//以下のライブラリをインポートしないと機能しない
import java.util.HashSet;
import java.util.Set;

//セットの作成
Set<String> colors = new HashSet<String>();

//データ挿入
colors.add("赤");
colors.add("青");
colors.add("黄");

//あえて重複したデータを挿入してみる
colors.add("赤");
```



```
//リストのサイズを確認する
System.out.println("色は" + colors.size() + "種類");
```

結果

色は3種類

要素の順番について

```
# 要素の順番は適当に取り出される
青->赤->黄->
```

Set関連のクラスはざっくりList型として使用するのが一般的

```
//例
Set<要素の型> Set変数名 = new HashSet<要素の型>();
Set<要素の型> Set変数名 = new LinkedHashSet<要素の型>();
Set<要素の型> Set変数名 = new TreeSet<要素の型>();
```

Map関連のクラス

1. 変数宣言

```
Map<キーの型, 値の型> Map変数名 = new HashMap<キーの型, 値の型>();
// ※ 右辺のインスタンス型名は省略可能
```

2. 要素の追加

```
Map変数名.put(キー, 値);
// ※ キーは重複不可
```

3. 要素の取り出し（拡張for文でkeyの集合から一個ずつキーを取り出しそれをもとにgetメソッドで値を取り出す）

```
for( キーの型 key : マップ変数.keySet() ) {
    値の型 value = マップ変数.get( key );
}
```

例文

```
//ハッシュマップの作成
Map<String, Integer> prefs = new HashMap<String, Integer>();

//データの挿入（キー、値）
prefs.put("京都府", 255);
prefs.put("東京都", 1261);
prefs.put("熊本県", 181);

//キーを指定して、データを取得
int tokyo = prefs.get("東京都");
System.out.println("東京都の人口は" + tokyo);

//キーの削除
prefs.remove("京都府");

//キーの更新
prefs.put("熊本県", 182);

int kumamoto = prefs.get("熊本県");
System.out.println("熊本県の人口は、" + kumamoto);

//県名を一覧取得する -> keySet()は、ハッシュ配列に入っている、キー名をすべて取得する
for(String key : prefs.keySet()) {
    int value = prefs.get(key);
    System.out.println(key + "の人口は" + value);
}
```

Mapそれぞれの特性

取り出される順番は順不同（順番は保証されない）

Mapを使う場合で順番をきちんと保証したい場合は、以下

- LinkedHashMapクラス ... 格納した順に取り出される
- TreeMapクラス ... 自然順序付け（abc順、あいうえお順）で取り出される

を使う。

※HashSetが順不同で、上はちゃんと順番がつくということ

Map関連のクラスはざっくりMap型として使用するのが一般的

```
//例

Map<要素の型> Map変数名 = new HashMap<要素の型>();
Map<要素の型> Map変数名 = new LinkedHashMap<要素の型>();
Map<要素の型> TreeMap変数名 = new TreeMap<要素の型>();
```

参照型による、アドレス渡し

リストはアドレスを渡しており、インスタンス名を変更するなどして、名前が変更できてしまう。

```
//ヒーローインスタンス作成
Hero h = new Hero();

//nameをミナトにする
h.name = "ミナト";

//listにheroインスタンスを登録する
List<Hero> list = new ArrayList<Hero>();
list.add(h);

//入れたものをさらに名前を変更
h.name = "スガワラ";

//参照型のリストのため、アドレスを格納したため、後から変えても変更できる
System.out.println(list.get(0).name);
```

結果

スガワラ

エラーの種類

- 文法エラー（syntax error） ... 文法の誤りによりコンパイルに失敗する。（セミicolon忘れ、変数名の間違いなど）

```
# 例文
Main.java:5: エラー: ';'がありません
    System.out.println("START")
                        ^
```

- 実行時エラー（runtime error） ... 実行している最中に何らかの異常事態が発生し、動作が継続できなくなるエラー。強制終了される。（配列の範囲外を指定、0で割る、存在しないファイルへのアクセスなど）

```
>java Main
プログラムを開始します。処理を3つ実行します。
処理 1 を完了。
処理 2 を完了。
```

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException:
Index 3 out of bounds for length 3
    at Main.main(Main.java:11)
```

- 論理エラー (logic error) ... Javaの文法に問題はなく、強制終了もしない。だが結果が自分の思っていたものと違っていた場合のエラー (3 + 5 = 30みたいな風に論理的にエラー)

プログラムを開始します。
3 + 5を計算します。
計算完了：答えは35
プログラムを正常に終了します

エラーメッセージごとの処理

Javaにおける例外処理の基本パターン

```
try {
    例外（エラー）が発生する可能性のある処理
} catch (例外 1 の型 変数名) {
    例外 1（エラー 1）が発生したときの処理
} catch (例外 2 の型 変数名) {
    例外 2（エラー 2）が発生したときの処理
} catch (...) {
    ...
}
```

例外処理（catch）すべきな処理はどのようなときか？

- Error系例外（致命的であり、回復することができない）
 - **OutOfMemoryError ... JVMのメモリ不足**
 - **ClassFormatError ... クラスを読み込むときに形式が壊れている**
- Exception系例外（回復見込みがあり、catchすべき）
 - **IOException ... ファイルやネットワーク入出力で問題が発生**

プログラム例

```
FileWriter fw = new FileWriter("data.txt");
```

- **ConnectException ... ネットワーク接続の失敗**
- RuntimeException（回復しなくてもいいレベル。catchしなくてもよい）
 - **NullPointerException ... Nullが入っているとき**
 - **ArrayIndexOutOfBoundsException ... 配列の範囲外にアクセスしてしまったとき**

例外インスタンスの利用

例外インスタンスが最初から持っているメソッド

- getMessage() ... エラーメッセージ取得
- printStackTrace() ... スタックトレース（エラーが発生したときのメソッドなどの呼び出し履歴）の内容を画面に出力する

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at com.example.MyClass.divide(MyClass.java:10)
    at com.example.MyClass.main(MyClass.java:5)
```

2つ以上の例外がある時の例外処理

```
try {
    FileWriter fw = new FileWriter("data.txt")
} catch (IOException | NullPointerException e) {
    処理
}
```

例外処理の作り方の例

```
try {
    FileWriter fw = new FileWriter("./data2.txt");
    fw.write("Hello!");
    fw.close();

} catch (Exception e) {
    System.out.println("何らかの例外が発生しました");
}
```

finallyについて

finallyは例外が発生しても必ず実行される場所

```
//ここではfwのスコープ範囲外でclose()しているため、エラー
try {
    FileWriter fw = new FileWriter("data.txt");
    fw.write("hello!");
}catch(Exception e) {
    System.out.println("何らかの例外が発生しました");
}finally {
    fw.close();
}

//正しい書き方 -> これでスコープエラーが消えるが、「何らかの例外が発生しました」が出てこない
FileWriter fw = null;
try {
    fw = new FileWriter("data.txt");
    fw.write("hello!");
}catch(Exception e) {
    System.out.println("何らかの例外が発生しました");
}finally {
    fw.close();
}
```

正しい書き方

```
//最初にnullを入れておく
FileWriter fw = null;

try {
    fw = new FileWriter("data.txt"); //ファイルを取り込み
    fw.write("hello!"); //書き込み処理
    throw new IOException(); //IOExceptionを投げる
}catch(IOException e) { //IOExceptionをキャッチ
    System.out.println("何らかの例外が発生しました" + e.getMessage()); //エラー処理
}finally {
    if(fw != null) { //ファイルを開いてなかったらそのまま、開いていたら閉じる
        try{ //ファイルを閉じる処理
            fw.close();
        }catch(IOException e){ //ここでもIOExceptionが発生したら止める
        }
    }
}

//試しに例外処理をぶちこみたい場合
throw new IOException(); //IOExceptionをぶちこむ
```

なぜfw.closeもtry_catchで囲まないといけないのか？

`FileWriter("data.txt")`と、`fw.close()`は必ず、`try_catch`で囲まないといけない。理由は、そのままにしておくと危ないという、Javaでの特殊な規約があり、それをコードで実装できていなかったらJVMが感知してコーディングエラーにさせるから。なので、必ず`try_catch`で囲まないとエラーになってしまう。

try_withリソース文

例外の伝播（でんぱ）

メソッド1がメソッド2を呼び、メソッド2がメソッド3を呼び。しかし、メソッド3でエラーが発生して、メソッド2にエラーを返し、メソッド2がそのエラーをそのままメソッド1に返す。このような流れを例外の伝播という。（Exception系を返していく）

IllegalArgumentを使って、エラーメッセージを扱う

```
// 例文

//年齢クラス定義 -> 年齢がマイナスだったらエラー
public class Person {
    int age;
    public void setAge(int age) {
        //引数チェック
        if(age < 0) {
            //マイナス時のエラーを呼び出し元に返す
            throw new IllegalArgumentException("年齢は0以上の数を指定すべきです。指定値="+
+ age);
        }
        //問題ないなら格納
        this.age = age;
    }
}

//メインクラス
public static void main(String[] args) {

    //もしもIllegalArgumentExceptionを検知した場合、処理
    try {
        Person p = new Person();
        p.setAge(-128); //ageに-128を渡してエラーにさせる
    }catch(IllegalArgumentException e) {
        System.out.println("エラー発生。終了します。");

        //ここにIllegalArgumentExceptionと書かれている
        System.out.println(e.getClass());

        //上のクラスで書いたメッセージがこの、getMessage()の中に入っている。
        System.out.println(e.getMessage());
    }
}
```

例外を作る

例外を作成し、試験的に例外を発生させることができる。

```
//Exceptionクラスを継承して、新たな例外クラスを作成する。
public class UnsupportedMusicFileException extends Exception{
    //エラーメッセージを受け取るコンストラクタ
    public UnsupportedMusicFileException(String msg) {
        super(msg); //継承元にエラーメッセージを引数として送る。なぜかエラークラスが自動で入力される
    }
}

//メインクラス
public static void main(String[] args) {
    try {
        //試験的に例外を発生させる
        throw new UnsupportedMusicFileException("未対応のファイルです。");
    }catch(Exception e) {

        //例外の詳細メッセージを出力するのが、printStackTrace()
        e.printStackTrace();
    }
}
```

eを出力するのと、e.printStackTraceを出力するのと、どう違うのか

- eだけ

eだけだとエラークラスと、エラーメッセージだけが出力される。

- e.printStackTrace()だと（スタックトレース）

e.printStackTraceだと、エラークラスとエラーメッセージ + 呼び出し元とエラーの場所を示してくれるから、こっちのほうが便利である。

TrueまたはFalseを返すメソッドについて

TrueまたはFalseを返すメソッドの名前は、末尾に?を付ける

デファクタースタンダードとは? -> 公的に定められた標準・基準

※TrueまたはFalseを返すメソッドに==Trueなどは、つけない。これはデファクタースタンダード

```
#if メソッド名
# 処理
#end
#とする

if logged_in?
```



```
    puts "ログインしています"
  end

  # current_userがnilじゃなかったらTrue
  def logged_in?
    !current_user.nil?
  end

  # current_user取得
  def current_user
    if session[:user_id] # セッションを持ってきて、あったらTrue
      @current_user ||= User.find_by(id: session[:user_id]) # もし@current_userが
      なかったら、セッションから持ってきたidでユーザを代入する
    end
  end
end
```

javaスクリプトとRailsの関係

Railsはサーバー上で動作し、Javaスクリプトはクライアントで動作する。

ボタンを押すとサブメニューが出たり、個数を入れると金額が出たりするブラウザ上の変化はJavaスクリプトで行う。

Railsの場合、import_mapという機能を使いjavaスクリプトの制御を行う。

import_mapの使い方

config/importmap.rbに使いたいJavaスクリプトライブラリと場所を登録